

One Timetable

A scheduling and room-booking system for IIT Delhi Abu Dhabi

Evan Johan Tobias

Draft 1 — 11 September 2026

Contents

Part 1 — The case	1
Part 2 — What exists, and what we should do about it	2
Part 3 — Scope	3
Part 4 — The domain model	4
Part 5 — The five flows	4
Part 6 — How it should look	5
Part 7 — Architecture	6
Part 8 — Rollout	7
Part 9 — Decisions management must make	8
Part 10 — What is already built	9
Appendix A — Sources	9
Appendix B — Reproducing the findings	9
Technical appendices	10
Appendix C — Schema	10
Appendix D — The validation catalogue	13
Appendix E — API surface	14
Appendix F — State machines	15
Appendix G — Screen specifications	16
Appendix H — Emitting iCalendar	17
Appendix I — Permissions	17
Appendix J — Testing	18
Appendix K — Effort	18

Part 1 — The case

1.1 What the current process is

The schedule lives in five Excel workbooks maintained in the academic office. Those are exported to PDF through Acrobat PDFMaker and uploaded to iitdabudhabi.ac.ae/timetable as eleven separate files, one per cohort. Everything downstream — students, faculty, room users — reads the PDFs.

There is no step between “someone edits a cell” and “eleven PDFs are correct and everyone knows”.

1.2 Defects this produces, measured

Running every published sheet and every workbook through a reader built for this purpose, on 10–11 September 2026:

Finding	Count	Example
Rooms called different names on different sheets	6	M4-0-019 is “Classroom 3”, “Classroom 5” and “Classroom 8”
Rooms double-booked across cohorts	1	Wed 17:00 M4-0-019: Masters AAIL7024 vs Y2 EEN AMLL1001
Blocks whose end time precedes their start	2	Y1 ELE and Y1 EEN, Monday AMLL1001: “15:30 to 14:20”
One class described two ways	1	AHUL261 Mon 09:00 — “Tutorial” on the CSE sheet, unlabelled on EEN
Room name contradicted by its own door plate	1	Y1 CHE calls M4-0-011 “Lecture Hall”; the plate says Classroom 3

Two of these compound: the Y2 EEN row in the double-booking prints code M4-0-019 beside the name “Classroom 8”, and Classroom 8 is M4-1-011 everywhere else. Either the code or the name is a typo, and which one it is decides whether the clash is real. Nobody can answer that from the documents.

1.3 The distribution problem, separately

Sheets are reissued often — 23 Aug, 27 Aug, 7 Sep, 8 Sep, 9 Sep, 10 Sep for various cohorts — and nothing announces it. A snapshot taken on 10 September found 5 of 11 held copies out of date, two of them by thirteen days. Those two carry no revision stamp in the footer at all, so even a careful reader comparing stamps sees nothing wrong.

The workbooks are also *ahead* of the website: Year 2 is dated 10 September while the newest published Year 2 PDF is the 8th.

1.4 What this costs

- A student walks to the wrong room, because the name on the sheet is not the name on the door.
- Two classes arrive at the same room, and the more junior group leaves.
- A change is published and reaches nobody until someone happens to re-download.
- The office cannot answer “is M4-0-019 free on Thursday at 3?” without opening eleven files and reading them by eye.

None of this is carelessness. It is what happens when eleven documents are the system of record and a spreadsheet is the only integrity check.

Part 2 — What exists, and what we should do about it

2.1 The market

System	Vendor	Shape	Fit here
25Live	CollegeNET	Cloud. Academic + event + resource scheduling, approval workflow, conflict checking. Market leader in North American HE.	Model is right, scale and cost are not
Syllabus Plus / Granular	Scientia	Capacity-aware section placement, automated timetabling	Enterprise; built for thousands of sections

System	Vendor	Shape	Fit here
CELCAT	CELCAT	Timetabling + room booking + SIS integration, strong UK presence	Same
Untis	Untis GmbH	Schools, mostly Europe, integrated SIS	Wrong segment
UniTime	Apereo Foundation	Open source, Java. Course + exam timetabling, student scheduling, room sharing, distributed departmental editing	Closest technical match; heavy

2.2 Buy, adopt, or build

Don't buy. These are priced and scoped for institutions with tens of thousands of students and dedicated scheduling offices. IITD-AD has eleven cohorts and one person maintaining the workbooks.

Don't adopt UniTime either, despite it being free. It is a large Java/Hibernate application designed for distributed departmental scheduling at a research university, with an automatic timetabling solver at its centre. We do not have the problem it solves. Our schedule is *already built* by a person who knows what she is doing; what fails is everything after that.

Build a focused tool, and copy 25Live's domain model rather than its software. That model is consistent across every serious product in the category:

request a space → rules-based approval → real-time conflict detection → publish

Four stages. We need all four, at a scale one person can operate.

2.3 The one thing worth taking from the standards, not the products

Emit iCalendar (RFC 5545). Do not implement CalDAV. A read-only .ics feed over HTTPS gets roughly 95% of interoperability value for a fraction of the work; CalDAV (RFC 4791 plus RFC 6638 scheduling) is what a full two-way calendar *server* implements and is enormous overkill for publishing.

And a finding that changes the design: Google Calendar refreshes subscribed ICS feeds every 12 to 24 hours, is not configurable, has no force-refresh, and ignores both REFRESH-INTERVAL (RFC 7986) and Microsoft's X-PUBLISHED-TTL. Apple Calendar and Thunderbird poll every 5–15 minutes; Outlook web about 3 hours. Google is both the slowest and the most used.

An ICS feed can never be the notification channel. A room change posted at 09:00 does not reach a Google subscriber until the next day. Split the responsibilities on day one: the feed is “what's on this month”, the web app is the source of truth for “is this still on and where”, and same-day changes go out through a separate push path.

Part 3 — Scope

3.1 In scope for v1

1. One authoritative schedule for every cohort, every term.
2. Room booking: request, approve, and see availability.
3. Publishing: web views, per-cohort and per-room, plus ICS feeds.
4. Conflict prevention at write time, not discovery after the fact.
5. Change notification with a human-readable diff.
6. Utilisation reporting.

3.2 Explicitly out of scope for v1

- Student course registration and add/drop. Real, wanted, and a different system. Staged as v3 in Part 8.
- Automatic timetable generation. Constraint solvers are the most expensive part of every product above and solve a problem the office does not have.
- Exam scheduling. Separate cycle, separate constraints.
- Attendance. Different system entirely.

Saying no to these in v1 is what makes v1 shippable.

Part 4 — The domain model

4.1 Entities

Term	2026–27 Sem 1, start/end, no-class days
Building	M1 M2 M3 M4
Room	code (identity), plate name, capacity, kind, building, floor
Course	code (identity), title, L–T–P–C, department
Cohort	24A1CSEBSEM5 -> "Y3 CSE Sem 5", size
Group	a cohort's teaching split: 1, 2, 3, 4, and sub-splits 1A, 2C
Section	a course taught to a cohort in a term, with an instructor
Meeting	a section in a room at a time, for a set of groups
Booking	a non-teaching use of a room, with a requester and a status
Person	staff, faculty, student; role-scoped

4.2 The rules that make it hold together

A room's code is its identity. Every name is an attribute. M4–0–019 is the room. “Classroom 5” is what the door says. “Classroom 3” is what one sheet claims. Both are stored, neither is the key, and the disagreement stays visible instead of being silently resolved. This single rule retires all six naming defects in §1.2.

Nothing references a course except by code. Not by title, not by short name. A course legitimately has several names — the official title, the everyday name, the abbreviation on a timetable pill — and each is a field on one object.

A meeting names its groups explicitly. [] means the whole cohort. [1] means group 1 only. This is exactly what the workbooks already encode as merged cell spans, and exactly what the PDF export destroys.

Every published state is versioned. A revision has a stamp, an author, and a diff against its predecessor. “What changed between the 7th and the 8th” must be a query, not an archaeology exercise.

Part 5 — The five flows

5.1 Build a term

The office keeps working in Excel. That is not a compromise, it is correct: the workbook is where the thinking happens, and forcing a grid editor on someone who is fluent in Excel buys nothing.

- 1 scheduler uploads the term workbook
- 2 system reads it: cohorts, meetings, groups from merge spans, the L–T–P–C table
- 3 system validates BEFORE accepting:
 - every course in the grid appears in the credit table, and vice versa
 - contact hours reconcile against L–T–P–C, per group
 - no end time precedes its start
 - every room code exists
 - no two meetings occupy one room at one time
- 4 clean -> staged, with a diff against the current published term

not clean → rejected, with the exact cells listed. Nothing is written.
 5 scheduler reviews the diff and publishes

Step 3 is the whole value. Both “15:30 to 14:20” errors and the double-booking would have been refused at upload.

5.2 Publish

One publish writes every view at once:

- per-cohort week (web)
- per-room week (web) — *what Priya asked for on 11 September, and does not exist today*
- per-instructor week (web)
- ICS feed per cohort, per room, per instructor
- a PDF export per cohort, byte-for-byte the layout people already know

The PDF export matters politically. It means adopting the system costs nobody their familiar artefact, and it removes the argument “but we need the PDF”.

5.3 Book a room

requester picks a room and a time

- system shows availability live, from the same table teaching uses
- submit creates a request in PENDING
- routing rule sends it to the approver for that room
- approver sees the request beside that room's day
- APPROVE writes a booking; the exclusion constraint refuses an overlap
- requester is notified; the room's views and feeds update

A booking and a class are the same kind of row. If they are separate tables, they will overlap eventually.

5.4 Change a published schedule

edit → validate → diff → publish → notify

The notification is a sentence, not a file:

Revision 6, 11 September 14:20. ACOL351's Wednesday tutorial moved from M4-0-019 to M4-0-011, same time. One other change: AHUL261's Monday tutorial is now 09:00, was 15:30. Nothing else changed.

The office already writes these by hand in email. Generate them: sets in, sets out, sets moved.

5.5 Report utilisation

The standard measures, which are what management actually asks for:

- Frequency rate — share of available hours a room is timetabled
- Occupancy rate — headcount against capacity while in use
- Utilisation — the two multiplied; the headline estate figure
- Show rate — enrolled against actually present, where attendance data exists

Nothing here needs sensors. Timetabled frequency alone answers “do we need more rooms” and “which rooms are never used”, and both are live questions on a campus this size.

Part 6 — How it should look

6.1 Principles

Show state in form, not only in colour. Provisional, confirmed, cancelled and pending must each be distinguishable without relying on hue.

Colour encodes kind, not identity. Lecture, tutorial, lab, reserved. Colouring by course produces a rainbow that means nothing at a glance and breaks with more than about eight courses.

Never render an unverified value as though it were verified. If a room name is disputed across sources, the view says so rather than picking silently.

6.2 The views

Cohort week — horizontal. Days down the side, time across the top, matching the orientation of the printed sheet people already know. Group badges on any block that does not apply to the whole cohort. This is the default landing view.

Room day — the booking surface. One room, one day, time across. Existing teaching in solid blocks, bookings in a lighter fill, free time genuinely empty and clickable. Dragging across free time opens a request pre-filled.

Room week grid — the office view. Rooms down the side, days across, each cell a compressed bar of what is in it. This is the one that makes a clash visible without anyone looking for it.

Availability finder. “I need a room for 90 minutes on Tuesday afternoon that seats 40.” Returns rooms, ranked by how well the capacity fits, because sending a seminar of 12 into a 120-seat lecture hall is a real cost.

Utilisation heatmap. Rooms against hours, colour by frequency. Dead rooms and pressure points both become obvious in one screen.

Mobile. Single column, and the top of the screen answers one question: what is next, where, and how long until it starts.

6.3 Theming

The institution’s own palette, taken from iitdabudhabi.ac.ae:

Primary	#A41E22	IITD crimson, the dominant brand colour on their site
Deep	#690F12	headers, hover states
Tint	#FCE6E6	selected rows, soft emphasis
Ink	#191919	primary text
Ink secondary	#2C2A2E	headings
Muted	#475569	labels, metadata
Rule	#E2E8F0	borders, grid lines
Ground	#F7F7F7	page background
Surface	#FFFFFF	cards, panels

Semantic colours are deliberately not crimson, so that “needs attention” never competes with brand:

OK	#2E7D5B	
Warning	#B4770F	
Critical	#B3261E	(distinct from brand crimson, used only for real errors)

Typography. Their site uses Inter and Matter. Inter is freely available and should carry the interface. Matter is a commercial licence — if the institution holds one, use it for headings; if not, Inter alone is correct and consistent.

A monospaced face for room codes, times and L-T-P-C values. These are data, they line up in columns, and M4–0–019 in proportional type is harder to scan.

Bilingual. Door plates are Arabic and English. Room labels in the interface should carry both.

Part 7 — Architecture

7.1 Stack

Database PostgreSQL

Backend	one application server; Python/FastAPI or Node/Fastify
Frontend	server-rendered pages plus light client JS
Auth	institutional SSO (Microsoft 365; campus already runs Outlook and Teams)
Hosting	on campus, on institution-owned infrastructure

Postgres is not interchangeable here. The core guarantee is a database constraint only it provides.

7.2 The constraint that does the real work

```
CREATE EXTENSION btree_gist;
```

```
CREATE TABLE occupancy (
  id          bigserial PRIMARY KEY,
  room_id    text      NOT NULL REFERENCES room(code),
  during     tstzrange NOT NULL,
  kind       text      NOT NULL,          -- 'class' | 'booking' | 'hold'
  status     text      NOT NULL DEFAULT 'confirmed',
  EXCLUDE USING gist (room_id WITH =, during WITH &&)
  WHERE (status <> 'cancelled')
);
```

Two requests arriving in the same millisecond both pass an application-level check; the constraint refuses the second row. This is the difference between “we try not to double-book” and “we cannot double-book”. Making it partial (`WHERE status <> 'cancelled'`) matters, or cancelled events hold their slots forever.

7.3 Roles

Viewer	public. Published schedules, room availability. No login.
Student	their own cohort surfaced first; subscribe to feeds.
Faculty	their own teaching; request rooms.
Approver	approve or refuse requests for rooms they own.
Scheduler	upload a term, edit meetings, publish revisions.
Admin	rooms, users, terms, approval routing.

7.4 Operational requirements

- Every write is attributed and timestamped. “Who moved this and when” must always have an answer.
- Nightly database backup, restore tested at least once before go-live.
- Read-only mode that keeps published views serving if the admin side is down.
- Full export to Excel at any time. The institution must never be locked into this tool, and saying so in writing is what makes it adoptable.

Part 8 — Rollout

Each phase is independently useful. If the project stops after any one of them, what was delivered still stands on its own.

Phase 0 — Publish what already exists (about 2 weeks)

Read-only. Import the current workbooks, publish cohort views, room views, and ICS feeds. No booking, no editing, no login.

Delivers: the room-wise schedule the office has already asked for; one URL that is always current; the end of the eleven-stale-PDFs problem. *Risk:* near zero. Nothing is written, nothing is replaced.

Phase 1 — Upload with validation (about 3 weeks)

The scheduler uploads a workbook; the system validates, shows a diff, publishes on approval. SSO and roles land here.

Delivers: the class of errors in §1.2 becomes impossible to publish.

Phase 2 — Room booking (about 4 weeks)

Requests, approval routing, the availability finder, the exclusion constraint.

Delivers: booking stops being an email thread.

Phase 3 — Change notification (about 2 weeks)

Generated diff paragraphs, email and web push to affected cohorts. Separate from the ICS path, for the reason in §2.3.

Phase 4 — Utilisation reporting (about 2 weeks)

The measures in §5.5, exportable.

Phase 5 — Course pages and add/drop (a term of work, decided separately)

Only once the schedule is trustworthy. The standard pattern is well established: a shopping cart students fill before registration opens, a validation pass checking prerequisites, credit limits, time conflicts and consent, then enrolment with waitlists where a section is full. This needs a decision about whether it integrates with the existing ERP or replaces part of it, which is Part 9.

Part 9 — Decisions management must make

These block design, not implementation. Each needs an owner and an answer.

1. Where does it run, and who owns it after I graduate? The requirement is campus-local, which rules out the public cloud. It needs a maintainer named before go-live. A system nobody can maintain in two years is worse than the spreadsheet.
 2. Does it become the system of record? If the office keeps editing workbooks and the system imports them, it is a publisher and the workbook stays authoritative. If the office edits in the system, it is the record and the workbook becomes an export. Phase 0 and 1 work either way; Phase 2 onward assumes the second.
 3. Who approves a booking, per room? Approval routing is a policy question, not a technical one, and it cannot be guessed.
 4. What integrates? Blackboard, the ERP, Outlook room resources. Each is a separate piece of work and none is required for Phase 0 to 3.
 5. Who may see what? Are schedules public, campus-only, or role-scoped? This determines whether Phase 0 needs authentication at all.
 6. Data ownership and exit. The institution owns the data and can export it in full at any time. Worth stating explicitly in the proposal rather than leaving implied.
-

Part 10 — What is already built

Not a greenfield proposal. Working today, in github.com/EchoRover/links:

- A reader that pulls every cohort’s schedule out of the office workbooks, taking group membership from merged cell spans. Verified exact against a hand-checked week on day, time, course, room, kind and group.
- A reader for the published PDFs, for the case where only a PDF exists, that refuses rather than guesses when a file is not machine-readable.
- The room-wise pivot in §5.2, which found the defects in §1.2.
- A live student-facing site (linkcs.vercel.app) carrying the Sem 5 timetable, free-room finder, campus bus times and a wall-display board.

Phase 0 is largely a matter of connecting what exists to a database and a URL.

Appendix A — Sources

- CollegeNET 25Live — collegenet.com/scheduling/25live
- UniTime, Apereo Foundation — apereo.org/programs/software/unitime, github.com/UniTime/unitime
- RFC 5545 iCalendar · RFC 4791 CalDAV · RFC 6638 CalDAV scheduling · RFC 7986 REFRESH-INTERVAL
- RFC 8984 JSCalendar, and draft-ietf-calext-jscalendar-icalendar — the two formats are expected to coexist; emit iCalendar
- Registration UX patterns: Johns Hopkins SIS, Michigan State SIS, Case Western SIS, Tufts SIS public documentation
- Space utilisation measures: standard HE frequency/occupancy/utilisation definitions
- Brand values read from iitdabudhabi.ac.ae/assets/css/new-style.css

Appendix B — Reproducing the findings

<code>python3 tools/read_xlsx.py</code>	every cohort, from the workbooks
<code>python3 tools/room_schedule.py</code>	the room pivot, clashes, naming conflicts
<code>python3 tools/parse_sheet.py --all</code>	the published PDFs
<code>python3 tools/review.py</code>	a parse, laid out like the sheet

Technical appendices

Everything below is implementation detail. Part 1 to 10 can be read without it.

Appendix C — Schema

Postgres. Written out because the constraints are the design, not an afterthought.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- ----- estate -----

CREATE TABLE building (
  code      text PRIMARY KEY,          -- 'M4'
  name      text NOT NULL,
  name_ar   text
);

CREATE TABLE room (
  code      text PRIMARY KEY,          -- 'M4-0-019'. THE identity.
  building  text NOT NULL REFERENCES building(code),
  floor     text NOT NULL,            -- 'G', '1F'
  plate_en  text,                    -- what the door says
  plate_ar  text,
  capacity  integer,
  kind      text NOT NULL             -- classroom | lab | hall | seminar
      CHECK (kind IN ('classroom', 'lab', 'hall', 'seminar', 'other')),
  bookable  boolean NOT NULL DEFAULT true,
  approver_id bigint REFERENCES person(id),
  plate_photo text,                  -- evidence beats every document
  verified_on date
);

-- A name a SOURCE claims for a room. Kept so disagreements stay visible
-- instead of one source silently winning.
CREATE TABLE room_name_claim (
  room_code text NOT NULL REFERENCES room(code),
  source     text NOT NULL,          -- '24A1CSEBSEM5', 'door plate'
  claimed    text NOT NULL,
  seen_on    date NOT NULL,
  PRIMARY KEY (room_code, source, claimed)
);

-- ----- calendar -----

CREATE TABLE term (
  id        text PRIMARY KEY,        -- '2026-27-sem1'
  label     text NOT NULL,
  starts    date NOT NULL,
  ends      date NOT NULL,
  CHECK (ends > starts)
);

CREATE TABLE no_class_day (
  term_id   text NOT NULL REFERENCES term(id),
```

```

    day          date NOT NULL,
    reason       text NOT NULL,
    PRIMARY KEY (term_id, day)
);

-- ----- curriculum -----

CREATE TABLE course (
    code         text PRIMARY KEY,           -- 'ACOL351'
    title        text NOT NULL,             -- official, off the sheet
    short        text,                      -- what a pill shows
    dept         text,
    l            integer NOT NULL DEFAULT 0, -- L-T-P-C, stored apart so it
    t            integer NOT NULL DEFAULT 0, -- can be reconciled against
    p            integer NOT NULL DEFAULT 0, -- scheduled minutes
    credits      numeric(4,2) NOT NULL DEFAULT 0
);

CREATE TABLE cohort (
    code         text PRIMARY KEY,           -- '24A1CSEBSEM5'
    label        text NOT NULL,             -- 'Y3 CSE Sem 5'
    term_id      text NOT NULL REFERENCES term(id),
    headcount    integer,
    n_groups     integer NOT NULL DEFAULT 2  -- Year 1 runs four
);

CREATE TABLE section (
    id           bigserial PRIMARY KEY,
    term_id      text NOT NULL REFERENCES term(id),
    course_code  text NOT NULL REFERENCES course(code),
    cohort_code  text NOT NULL REFERENCES cohort(code),
    instructor   bigint REFERENCES person(id),
    UNIQUE (term_id, course_code, cohort_code)
);

-- ----- the one occupancy table -----
-- A class and a booking are the SAME kind of row. Separate tables would
-- eventually overlap, because nothing would stop them.

CREATE TABLE occupancy (
    id           bigserial PRIMARY KEY,
    revision_id  bigint NOT NULL REFERENCES revision(id),
    room_code    text NOT NULL REFERENCES room(code),
    during       tstzrange NOT NULL,
    kind         text NOT NULL
        CHECK (kind IN ('lecture', 'tut', 'lab', 'proj', 'help', 'booking', 'hold')),
    status       text NOT NULL DEFAULT 'confirmed'
        CHECK (status IN ('confirmed', 'provisional', 'cancelled')),
    section_id   bigint REFERENCES section(id),      -- set for teaching
    booking_id   bigint REFERENCES booking(id),      -- set for a booking
    groups       int[] NOT NULL DEFAULT '{}',        -- {} = whole cohort
    note         text,

    CHECK (num_nonnulls(section_id, booking_id) = 1),
    CHECK (upper(during) > lower(during)),

```

```

    EXCLUDE USING gist (room_code WITH =, during WITH &&)
    WHERE (status <> 'cancelled')
);

```

```

CREATE INDEX ON occupancy USING gist (during);
CREATE INDEX ON occupancy (section_id);

```

----- bookings -----

```

CREATE TABLE booking (
    id          bigserial PRIMARY KEY,
    requester_id bigint NOT NULL REFERENCES person(id),
    purpose     text NOT NULL,
    attendees   integer,
    status      text NOT NULL DEFAULT 'pending'
                CHECK (status IN ('pending', 'approved', 'refused',
                                'withdrawn', 'cancelled')),
    decided_by  bigint REFERENCES person(id),
    decided_at  timestampz,
    reason      text,
    created_at  timestampz NOT NULL DEFAULT now()
);

```

----- versioning -----

```

CREATE TABLE revision (
    id          bigserial PRIMARY KEY,
    term_id     text NOT NULL REFERENCES term(id),
    n           integer NOT NULL,
    source      text,
    source_hash text,
    stamp       text,
    author_id   bigint NOT NULL REFERENCES person(id),
    published_at timestampz,
    summary     text,
    UNIQUE (term_id, n)
);

```

```

CREATE TABLE person (
    id          bigserial PRIMARY KEY,
    sso_subject text UNIQUE NOT NULL,
    name        text NOT NULL,
    email       text NOT NULL,
    role        text NOT NULL
                CHECK (role IN ('student', 'faculty', 'approver', 'scheduler', 'admin')),
    cohort_code text REFERENCES cohort(code),
    active      boolean NOT NULL DEFAULT true
);

```

```

CREATE TABLE audit (
    id          bigserial PRIMARY KEY,
    at          timestampz NOT NULL DEFAULT now(),
    actor_id    bigint REFERENCES person(id),
    action      text NOT NULL,

```

```

entity      text NOT NULL,
entity_id   text NOT NULL,
before      jsonb,
after       jsonb
);

```

Why **occupancy** carries **revision_id**. Publishing a revision writes a new set of rows rather than mutating the old ones. "What did the timetable say on 8 September" is then a query, and a bad publish is reverted by pointing the term at the previous revision rather than by restoring a backup.

Appendix D — The validation catalogue

Every rule an upload must pass. The numbering is the error code the interface shows.

Structural (the file itself)

Code	Rule	Caught in the real data?
S1	Every sheet has a recognisable day column	—
S2	Every day band has group labels in column B	—
S3	A parsed sheet yields more than zero blocks	Yes. Both MTech sheets yielded zero and an earlier checker called that "ok"
S4	The workbook carries a term the system knows	—

Per block

Code	Rule	Caught?
B1	End time is strictly after start time	Yes. Y1 ELE and Y1 EEN: "15:30 to 14:20"
B2	Times fall inside the institution's teaching day (07:00–21:00)	—
B3	Room code parses and exists in room	—
B4	Course code exists in course	—
B5	The block's groups are a subset of its cohort's groups	—
B6	Duration is a plausible multiple (50 / 80 / 110 / 170 min)	—

Cross-block

Code	Rule	Caught?
X1	No room holds two different courses at overlapping times	Yes. Wed 17:00, M4-0-019
X2	No cohort-group is in two places at once	—
X3	No instructor is in two places at once	—

Code	Rule	Caught?
X4	A course taught to several cohorts at one time and place is ONE entry	Yes. 248 cohort entries collapse to 161 bookings; without this, 87 false clashes

Curriculum

Code	Rule	Caught?
C1	Every course on the grid appears in the L-T-P-C table	—
C2	Every course in the L-T-P-C table appears on the grid	—
C3	Scheduled contact hours reconcile against L-T-P-C, per group	Yes. This is what proves group membership
C4	Declared exceptions are listed, not inferred	AC0D310 is 0-0-6-3 with one slot reserved
C5	Non-contact blocks are excluded from C3	Yes. “Help Session if required” broke the reconciliation until excluded

Naming

Code	Rule	Caught?
N1	A room code maps to one plate name across all sheets	Yes. Six rooms fail this
N2	A plate name maps to one room code	Yes. “Classroom 8” is both M4-0-019 and M4-1-011
N3	Where a photographed plate exists, it wins, and disagreement is reported	Yes. Y1 CHE calls M4-0-011 “Lecture Hall”; the plate says Classroom 3

Severity matters. S and B failures *reject* the upload. X and C failures reject unless the scheduler explicitly acknowledges each one, because a genuine timetable can carry a deliberate overlap. N failures are *warnings* that publish: a naming disagreement should be visible, not blocking.

Appendix E — API surface

REST, JSON, thin. Public reads need no auth; everything else is SSO-gated.

Public reads

GET /api/terms	the terms, current first
GET /api/cohorts	every cohort, with labels
GET /api/cohorts/{code}/week?revision=	meetings, grouped by day
GET /api/rooms	every room, plate names, capacity
GET /api/rooms/{code}/week?from=	occupancy for a room
GET /api/rooms/free?day=&from=&to=&seats=	the availability finder
GET /api/instructors/{id}/week	
GET /ics/cohort/{code}.ics	RFC 5545 feed

```

GET /ics/room/{code}.ics
GET /ics/instructor/{id}.ics
GET /api/revisions?term=          revision history with diff summaries
GET /api/revisions/{n}/diff      sets in, out, moved

```

Authenticated

```

POST /api/bookings              create a request
    {room_code, during, purpose, attendees}
GET /api/bookings?status=&mine=
POST /api/bookings/{id}/approve
POST /api/bookings/{id}/refuse   {reason} -- required
POST /api/bookings/{id}/withdraw

POST /api/terms/{id}/uploads     multipart, the workbook
    -> 202 {upload_id}
GET /api/uploads/{id}           {status, findings[], diff}
POST /api/uploads/{id}/publish   {acknowledge: [...]}

GET /api/reports/utilisation?term=&from=&to=
GET /api/reports/naming-conflicts
GET /api/export/term/{id}.xlsx   full export, any time

```

Response shape, `/api/cohorts/{code}/week`

```

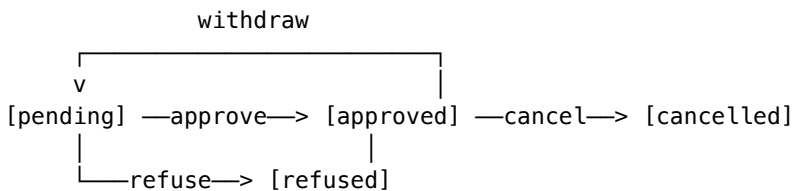
{
  "cohort": {"code": "24A1CSEBSEM5", "label": "Y3 CSE Sem 5", "n_groups": 2},
  "term":   {"id": "2026-27-sem1", "starts": "2026-08-20", "ends": "2026-12-16"},
  "revision": {"n": 6, "published_at": "2026-09-08T09:30:00Z",
               "stamp": "8th September 2026 - 1:30pm"},
  "meetings": [
    {"day": "Wednesday", "start": "15:30", "end": "16:20",
      "course": {"code": "ACOL351", "short": "Algos"},
      "room": {"code": "M4-0-019", "plate": "Classroom 5",
               "plate_disputed": true, "also_called": ["Classroom 3", "Classroom 8"]},
      "kind": "tut", "groups": [], "status": "confirmed"}
  ]
}

```

`plate_disputed` is not decoration. It is the interface's licence to show the reader that the documents disagree, instead of picking one and being confidently wrong.

Appendix F — State machines

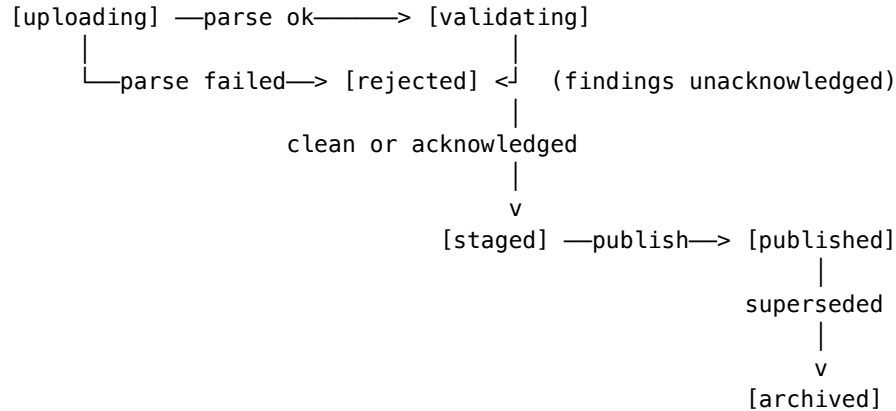
Booking



- `approve` writes an occupancy row inside the same transaction. If the exclusion constraint fires, the approval fails and the approver is told which booking holds the slot.
- `refuse` requires a reason. A refusal with no reason generates a follow-up email to the approver anyway, so the field may as well be mandatory.

- cancelled keeps the row. The slot is released by the partial index, and the history survives.

Revision



Only one revision per term is published. Rolling back is publishing an older revision, which creates a new revision row recording that it happened.

Appendix G — Screen specifications

G1 — Cohort week (the landing view)

- Layout. Days as rows, time 08:00–19:00 across. Matches the printed sheet’s orientation, which is the artefact everyone already reads.
- Block. Course code in mono at 11px; kind badge; group badge only when the block does not apply to the whole cohort; room plate; time. When a block is narrow, the badges survive and the room is what truncates — the badges are what the reader is checking.
- Overlap. Blocks that overlap stack into lanes. Nothing is ever hidden behind anything else.
- State. provisional draws a dashed outline plus a marker; colour alone never carries it.
- Now. A hairline at the current time on today’s row, and only on today’s row.
- Empty. A term with no published revision says so and links to the upload screen if the viewer is a scheduler. It never renders an empty grid.

G2 — Room day (the booking surface)

- One room, one day, time across the full width.
- Teaching in solid fill, approved bookings in a lighter fill of the same hue, pending requests hatched.
- Free time is genuinely empty and is the click target. Drag across it to open a request with times pre-filled.
- A right rail shows the room: plate name, capacity, kind, approver, and — if the plate is disputed — every name the documents give it.

G3 — Room week grid (the office view)

- Rooms down the side, weekdays across, each cell a compressed strip.
- This is the screen where a clash is visible without anyone searching for one.
- Sort by building, then floor, then code. Not alphabetically: physical adjacency is what the reader is thinking about.

G4 — Availability finder

I need a room [90] minutes on [Tuesday] after [14:00]
for [40] people in [any building ▼] [Find]

Results ranked by capacity fit, closest first, not alphabetically. Putting a seminar of twelve into a 120-seat lecture hall is a real cost, and a list sorted by room code invites exactly that.

G5 — Utilisation

- Rooms against hours, cell colour by frequency rate.
- A sequential ramp, not a rainbow. Dark equals busy.
- Below it, the three headline numbers per building: frequency, occupancy, utilisation.
- A “never used” list, which is the finding management acts on.

G6 — Mobile

Single column. The top of the screen answers one question: what is next, where, and how long until it starts. Everything else is below the fold and that is correct — the phone case is a person walking between buildings.

Appendix H — Emitting iCalendar

```
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//IITD Abu Dhabi//One Timetable//EN
CALSCALE:GREGORIAN
METHOD:PUBLISH
X-WR-CALNAME:Y3 CSE Sem 5
REFRESH-INTERVAL;VALUE=DURATION:PT1H
X-PUBLISHED-TTL:PT1H
BEGIN:VEVENT
UID:meet-8842@timetable.iitdabudhabi.ac.ae
DTSTAMP:20260908T093000Z
DTSTART;TZID=Asia/Dubai:20260909T153000
DTEND;TZID=Asia/Dubai:20260909T162000
RRULE:FREQ=WEEKLY;BYDAY=WE;UNTIL=20261216T235900Z
EXDATE;TZID=Asia/Dubai:20261028T153000
SUMMARY:ACOL351 Tutorial
LOCATION:M4-0-019 (Classroom 5)
DESCRIPTION:Analysis and Design of Algorithms\nTutorial\nAll groups
CATEGORIES:TUTORIAL
SEQUENCE:3
END:VEVENT
END:VCALENDAR
```

Points that are easy to get wrong:

- **UID** is stable for the life of the meeting. Regenerating it on every publish makes every subscriber’s calendar duplicate the entire term.
- **SEQUENCE** increments on change, so clients update rather than duplicate.
- Store local time with **TZID**, never bare UTC. 15:30 Asia/Dubai and its UTC instant are different facts, and if the term shifts the local time is what was meant. This is the specific mistake JSCalendar (RFC 8984) was designed to fix.
- No-class days become **EXDATE**, not deleted occurrences.
- Set **REFRESH-INTERVAL** and **X-PUBLISHED-TTL** anyway. They cost nothing and the clients that honour them are the ones that poll fast. Google ignores both.

Appendix I — Permissions

Action	Viewer	Student	Faculty	Approver	Scheduler	Admin
See published schedules	•	•	•	•	•	•
Subscribe to a feed	•	•	•	•	•	•
See room availability	•	•	•	•	•	•
Request a room			•	•	•	•
See who requested				•	•	•
Approve or refuse				• ¹	•	•
Upload a term					•	•
Publish a revision					•	•
Roll back a revision						•
Edit rooms and people						•
Read the audit log					•	•

* Only for rooms where they are the named approver.

Students cannot request rooms in v1. That is a policy question for Part 9, not a technical limit — the row exists, the permission is simply off.

Appendix J — Testing

Scale here is tiny: eleven cohorts, about 300 meetings, nineteen rooms. Load is not the risk. Correctness is. The test effort goes almost entirely into the validator and the importer.

Golden-file tests. The Sem 5 CSE week is hand-verified against the sheet and against its own credit table. It is the fixture: import it, assert 23 meetings exactly, on day, time, course, room, kind and group. Any importer change that breaks it is wrong.

Property tests. For any generated set of meetings, publishing then exporting then re-importing returns the same set. Round-tripping is where importers rot.

Constraint tests. Two concurrent approvals for the same slot: exactly one succeeds. Run it against a real Postgres, not a mock — the guarantee is the database's, so a mock tests nothing.

Regression corpus. Every defective sheet found this week becomes a fixture: the “15:30 to 14:20” file, the zero-block MTech sheets, the M4-0-019 clash, the “Classroom 8” mislabel. Each asserts the specific code from Appendix D.

What not to test. Rendering. Screenshot tests on a timetable grid are famously brittle and catch almost nothing that matters here.

Appendix K — Effort

Assuming one developer, part-time alongside coursework. Ranges, not promises.

Phase	Work	Estimate
0	Schema, importer, public read views, ICS	2–3 weeks
1	SSO, roles, upload with validation, diff and publish	3–4 weeks
2	Bookings, approval routing, availability finder	4–5 weeks
3	Change notification, generated diff paragraphs	2 weeks
4	Utilisation reporting	2 weeks
5	Course pages, add/drop	a term, scoped separately

Phase 0 is short because much of it exists. The importer that reads the office workbooks — including group membership from merged cell spans — is written and verified. The room pivot is written. A student-facing site carrying the Sem 5 timetable, a free-room finder and a wall-display board is live.

The honest risk is not the code. It is Part 9, question 1: who owns this after I graduate. Every estimate above assumes that gets answered before Phase 2, because Phase 2 is where the institution starts depending on it.